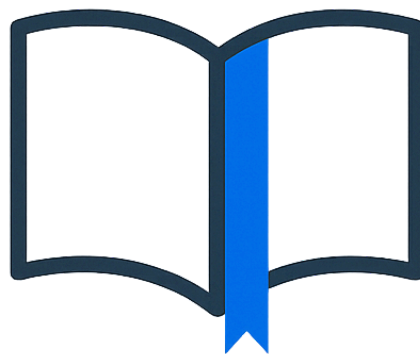


BookMarker
System Design Document
Versione 1.6



BookMarker

Data: 27/01/2026

Progetto: BookMarker	Versione: 1.6
Documento: System Design Document	Data: 27/01/2026

Coordinatore del progetto:

Nome	Matricola

Partecipanti:

Nome	Matricola
Francesco Liguori	0512109651
Antonio Prisco	0512123502
Nicholas Franco Grimaldi	0512109702

Scritto da:	Tutti i partecipanti
--------------------	----------------------

Revision History

Data	Versione	Descrizione	Autore
19/11/2025	0.1	Prima stesura	Tutti i partecipanti
21/11/2025	0.3	System Decomposition	Tutti i partecipanti
21/11/2025	0.4	Software Control	Antonio Prisco
25/11/2025	0.4.1	Gestione delle risorse globali	Francesco Liguori
25/11/2025	0.5	Boundary Conditions - Glossario	Nicholas Franco Grimaldi
05/12/2025	1.0	Aggiunto matrice degli accessi e specifica dei servizi dei sottosistemi, modificato System Decomposition	Tutti i partecipanti
27/01/2026	1.5	Aggiornata Matrice degli accessi, aggiornata Specifica dei servizi dei sottosistemi	Antonio Prisco, Francesco Liguori
27/01/2026	1.6	Modifiche all'indice e ai primi paragrafi	Nicholas Franco Grimaldi

Indice

1. Introduzione	4
1.1 Scopo del sistema	4
1.2 Obiettivi di design	4
1.2.1 Obiettivi di prestazioni	4
1.2.2 Obiettivi di usabilità	4
1.2.3 Obiettivi di affidabilità	4
1.2.4 Obiettivi di supportabilità	5
1.2.5 Obiettivi di manutenibilità	5
1.3 Definizioni, acronimi e abbreviazioni	5
1.4 Riferimenti	5
1.5 Panoramica	6
2. Architettura software corrente	6
3. Architettura software proposta	6
3.1 Panoramica	6
3.2 Decomposizione in sottosistemi	6
3.3 Hardware/Software mapping	8
3.4 Gestione dei dati persistenti	9
3.5 Access control & Security	12
3.6 Global Software Control	13
3.7 Boundary conditions	14
4. Specifica dei servizi dei sottosistemi	16

1.Introduzione

1.1 Scopo del sistema

La piattaforma BookMarker si propone come tramite tra la biblioteca e l'utente. Il sito permette di controllare in tempo reale la disponibilità attuale (o futura) dei libri presenti nel catalogo offerto dalla biblioteca. L'utente potrà avere il proprio account da "lettore", attraverso il quale potrà controllare la disponibilità attuale dei libri in biblioteca e prenotarne una copia da ritirare entro 3 giorni. Sarà inoltre possibile controllare: i libri attualmente presi in prestito, la relativa data di restituzione, e lo storico dei libri presi in prestito in precedenza. In aggiunta, per l'utente sarà possibile dare un voto e scrivere una recensione sui libri che ha preso in prestito e restituito, e potrà leggere le recensioni scritte da altri utenti su un particolare libro. Il bibliotecario avrà il compito di gestire le richieste di registrazione dei nuovi utenti sul sito, di gestire i prestiti degli utenti, prima gestendo le richieste di prenotazione e poi aggiornando gli stati del prestito nelle apposite sezioni. Avrà il compito di gestire il catalogo, aggiungendo e rimuovendo libri e aggiornando la loro disponibilità. Il moderatore avrà mansioni relative alle recensioni degli utenti e le rispettive segnalazioni: potrà nascondere, mostrare ed eliminare le recensioni e si occuperà di gestire le segnalazioni degli utenti.

1.2 Obiettivi di design

1.2.1 Obiettivi di prestazioni

- OP_01
I tempi di risposta della piattaforma devono essere inferiori a 1 secondo
- OP_02
Il sistema dev'essere in grado di gestire almeno 100 utenti contemporaneamente senza degradazioni evidenti delle prestazioni.

1.2.2 Obiettivi di usabilità

- OU_01
Il sito deve adattarsi automaticamente alle dimensioni dello schermo dal quale viene utilizzato.

1.2.3 Obiettivi di affidabilità

- OA_01
Le password sono memorizzate con hashing tramite jbcrypt-0.4 .
- OA_02
Il sistema è protetto da eventuali tentativi di SQL injection.

1.2.4 Obiettivi di supportabilità

- OS_01
Il sito può essere usato senza problemi su più browser diversi quali Google Chrome, Firefox, Brave e Microsoft Edge.

1.2.5 Obiettivi di manutenibilità

- OM_01
Il sistema è progettato in modo da garantire la futura manutenibilità, modularità e leggibilità del codice per facilitare la comprensione a chi dovrà intervenire su di esso

1.3 Definizioni, acronimi e abbreviazioni

Termine	Significato
BC-UC	Boundary Condition - Use Case
MySql	Gestionale del Database
XML	File di configurazione web
Apache Tomcat	Server Machine che gestisce logica applicativa e connessione al database
MVC	Model-View-Controller
PS	Problem Statement
RAD	Requirements Analysis Document

1.4 Riferimenti

- Problem Statement
- Requirements Analysis Document

1.5 Panoramica

Il seguente documento di system design descrive i dettagli di sviluppo del nostro sistema, "BookMarker". Gli obiettivi del sistema sono descritti generalmente nel documento di PS, sono invece descritte nel RAD tutte le funzionalità del sistema. Nel seguente documento si esporrà l'architettura software proposta, ovvero: decomposizione in sottosistemi, mappatura hardware/software, gestione dei dati persistenti, matrice degli accessi, specifica dei servizi dei sottosistemi e boundary conditions.

2. Architettura software corrente

Il sistema deve ancora essere sviluppato.

3. Architettura software proposta

3.1 Panoramica

Per il nostro sistema abbiamo deciso di utilizzare l'architettura Model View Controller. Questo per separare la logica dell'applicazione da ciò che l'utente vede e come interagisce con esso. Esso prevede una divisione in tre sottosistemi principali:

- Model: sottosistema responsabile della conoscenza del dominio applicativo. Si occupa della gestione dei dati, quindi della memorizzazione sul database e dell'interazione con stesso.
- View: sottosistema responsabile di mostrare gli oggetti del dominio applicativo agli utenti.
- Controller: sottosistema responsabile della sequenza di interazioni con l'utente e di notificare il View dei cambiamenti nel Model.

3.2 Decomposizione in sottosistemi

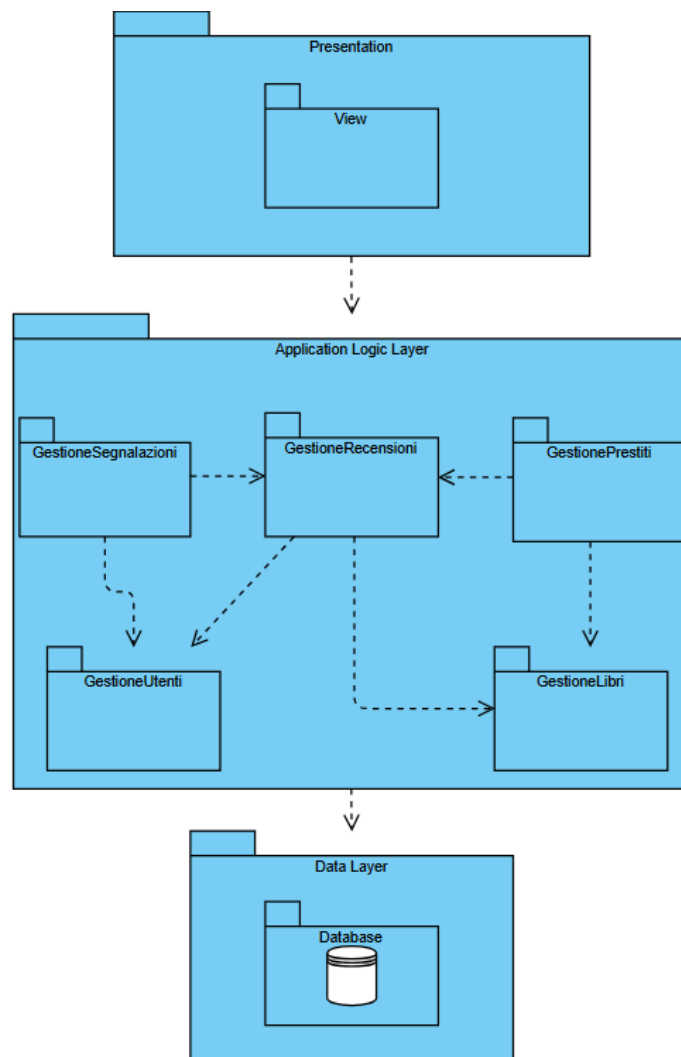
Il sistema è stato decomposto secondo lo stile Three-Tier.

Il Presentation layer contiene il sottosistema "View", responsabile dell'interazione con l'utente e della visualizzazione dei dati. Questo livello dipende dal layer sottostante per l'elaborazione delle richieste.

L'Application logic layer implementa la logica di business ed è diviso in cinque sottosistemi:

- GestioneUtente e GestioneLibri: rappresentano i moduli che gestiscono le entità fondamentali del sistema quali utenti e libri.
- GestionePrestiti e GestioneRecensioni: questi sottosistemi dipendono da GestioneUtente e GestioneLibri, in quanto un prestito e una recensione sono identificati dall'associazione tra un utente e un libro
- GestioneSegnalazioni: questo sottosistema dipende da GestioneUtente e GestioneRecensione, in quanto una segnalazione è identificata da un utente e una recensione.

Il Data layer contiene il database, responsabile della persistenza dei dati. Tutti i sottosistemi del livello applicativo dipendono da questo layer per il salvataggio dei dati e il recupero delle informazioni. La decomposizione scelta mira a minimizzare l'accoppiamento tra i sottosistemi e permette di avere un'alta coesione nei sottosistemi.



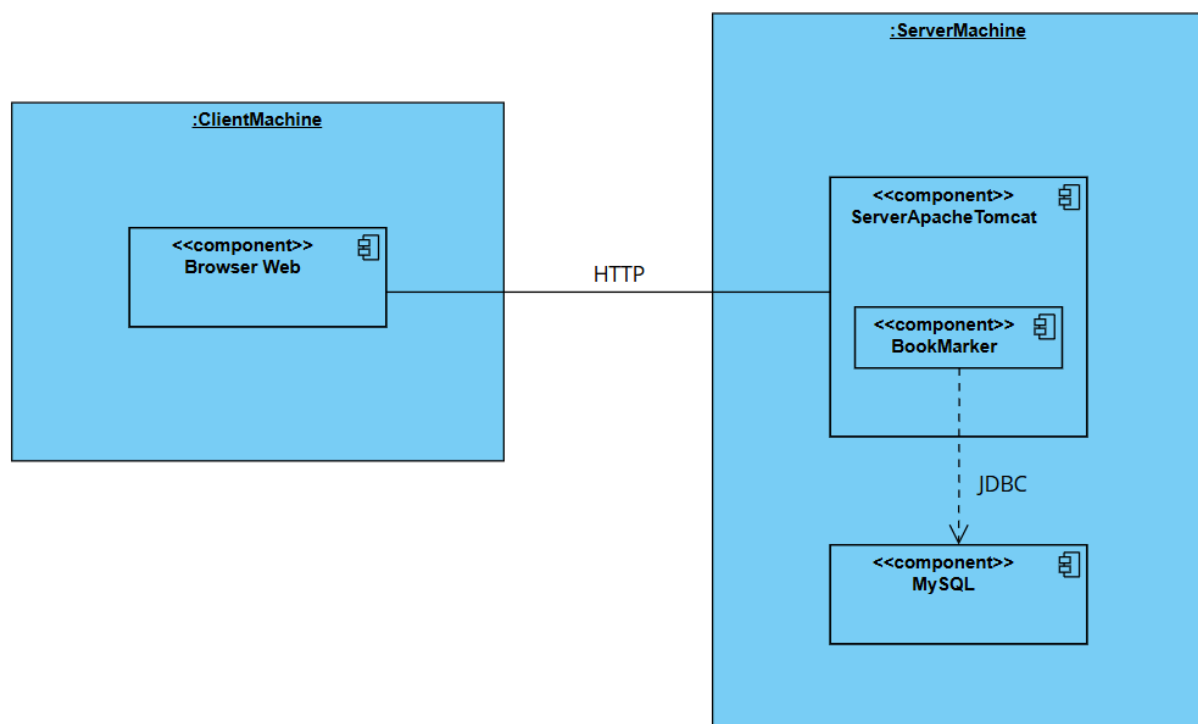
3.3 Hardware/Software mapping

L'architettura del sistema è di tipo MVC, basata su tecnologie web. Il sistema è distribuito su un Application Server (Apache Tomcat 9) che ospita la logica applicativa e comunica con un DBMS relazionale (MySQL) per la persistenza dei dati.

La comunicazione tra client e il server avviene tramite protocollo HTTPS.

Il deployment dell'applicazione viene effettuato sul server Apache Tomcat 9.

Sulla stessa macchina risiede l'istanza del DBMS MySQL (l'applicazione comunica con il database attraverso un driver JDBC).



3.4 Gestione dei dati persistenti

I dati del sistema che devono essere resi persistenti, delegati a un'istanza di MySQL, sono:

- Le informazioni degli utenti
- Le informazioni dei libri
- Le informazioni sui prestiti
- Le informazioni sulle recensioni
- Le informazioni sulle segnalazioni

Libri

Colonne	Valori
id_libro	int AI PK
titolo	varchar(100)
autore	varchar(100)
genere	varchar(100)
disponibilita	int
data_pubblicazione	date
descrizione	text
copertina	varchar(255)
data_rientro	date
attivo	tinyint(1)

Prestiti

Colonne	Valori
id	int AI PK
utente_email	varchar(100)
libro_id	int
data_prenotazione	date
data_inizio	date
data_fine_prevista	date
data_restituzione_effettiva	date
stato	varchar(20)
motivazione	varchar(255)

Recensioni

Colonne	Valori
id	int AI PK
utente_email	varchar(100)
libro_id	int
testo	text
data_inserimento	date
voto	int
visibile	tinyint(1)
eliminata	tinyint(1)

Segnalazioni

Colonne	Valori
id	int AI PK
utente_email	varchar(100)
recensione_id	int
motivo	varchar(255)
data_segna1azione	date
stato	varchar(20)
note_chiusura	text

Utenti

Colonne	Valori
nome	varchar(100)
cognome	varchar(100)
codice_fiscale	varchar(16)
email	varchar(100) PK
password	varchar(255)
data_registrazione	timestamp
recapito	varchar(100)
ruolo	varchar(20)
stato	varchar(20)
domanda_sicurezza	varchar(255)
risposta_sicurezza	varchar(255)

3.5 Access control & Security

Matrice degli accessi

Attori Oggetti	Visitatore	Lettore	Bibliotecario	Moderatore
Utente	registraUtente,	login, getDatiUtente, resetPassword	login, getUtentiDaApprovare, accettaUtente, rifiutaUtente, resetPassword	login, resetPassword
Libro	getCatalogoCompleto, getDettaglioLibro	getCatalogoCompleto, getDettaglioLibro	aggiungiLibro, aggiornaDisponibilita, rimuoviLibro, getCatalogoCompleto	getCatalogoCompleto, getDettaglioLibro
Prestito	-	prenotaLibro, getStoricoUtente	getPrenotati, getAttivi, getRestituiti, approvaRichiestaPrestito, rifiutaRichiestaPrestito, confermaRitiro, annullaPrestito, registraRestituzione, getRichiesteInAttesa	-
Recensione	getRecensioniPubbliche	aggiungiRecensione, getRecensioniPubbliche, getMappaRecensioniPerStorico, deleteRecensioneUtente	-	getRecensioniPerModeratore, deleteRecensioneModeratore, impostaVisibilita
Segnalazione	-	segnalaRecensione	-	getTutteLeSegnalazioni, chiudiSegnalazione

3.6 Global Software Control

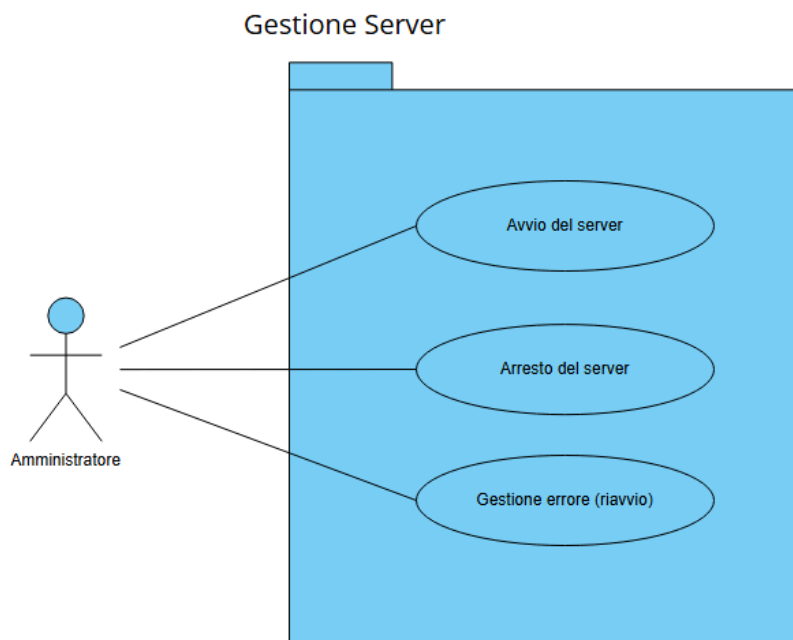
Il flusso di controllo globale è di tipo Event-driven. Questo per la natura interattiva dell'applicazione web, dove il controllo viene attivato da input esterni dell'utente.

Il meccanismo di controllo segue il pattern architetturale Model-View-Controller (MVC). Il flusso di esecuzione avviene nel seguente modo:

1. Il Web Server inizialmente attende gli eventi esterni, questi eventi corrispondono alle richieste HTTPS generate dalle interazioni dell'utente sull'interfaccia grafica (View).
2. Il dispatcher smista la richiesta allo specifico oggetto di controllo competente per quella funzionalità (es. ControllerRegistrazione, ControllerAutenticazione, ControllerCatalogo), come definito nei diagrammi di sequenza.
3. Il Controller esegue la logica applicativa necessaria interagendo con il Model.
 - Comunica con le classi di gestione (es. LibroService, UtenteService, etc) per le operazioni di ricerca, inserimento o recupero dei dati.
 - Accede e aggiorna lo stato delle singole entità dati (es. Libro, Prestito) una volta recuperate.

3.7 Boundary conditions

Le boundary conditions descrivono il comportamento del sistema in fasi particolari come la gestione, l'avvio, lo spegnimento, e la gestione degli errori. In questa sezione, abbiamo scelto di modellarli come casi d'uso:



BC-UC_1 Avvio del server

Attore: Amministratore

Entry Condition: L'amministratore si trova nella scheda di apache tomcat

Flusso di eventi:

1. L'amministratore avvia il server apache tomcat
2. Il sistema legge i file di configurazione xml
3. Il sistema si connette al database MySql
4. Il sistema inizializza i servizi dell'application logic layer

Exit Condition: Il sistema ora è avviato e accessibile agli utenti e l'amministratore si trova sulla scheda di apache tomcat

Flussi alternativi / Eccezioni:

BC-UC_2 Arresto del server

Attore: Amministratore

Entry Condition: L'amministratore si trova nella scheda di apache tomcat

Flusso di eventi:

1. L'amministratore esegue lo stop del server apache tomcat
2. Il sistema termina le sessioni utente attive
3. Il sistema chiude la connessione al database MySql

Exit Condition: Il sistema ora è spento e l'amministratore si trova sulla scheda di apache tomcat

Flussi alternativi / Eccezioni:

BC-UC_3 Gestione errore (riavvio)

Attore: Amministratore

Entry Condition: L'amministratore si trova nella scheda di apache tomcat

Flusso di eventi:

1. L'amministratore esegue il riavvio del server apache tomcat
2. Il sistema termina le sessioni utente attive
3. Il sistema chiude la connessione al database MySql
4. Il sistema rilegge i file di configurazione xml
5. Il sistema si riconnette al database MySql
6. Il sistema reinizializza i servizi dell'application logic layer

Exit Condition: Il sistema ora è avviato e accessibile agli utenti e l'amministratore si trova sulla scheda di apache tomcat

Flussi alternativi / Eccezioni:

Se al punto 6 il sistema è ancora in errore, l'eventuale eccezione sarà catturata dal sistema e visibile all'amministratore, e il sistema mostrerà ad un eventuale utente una pagina di "Manutenzione in corso" (BC-UC 3.1 Gestione errore fallita)

4. Specifica dei servizi dei sottosistemi

Gestione Utenti

Operazione	Descrizione
registraUtente(String nome, String cognome, String cf, String email, String password, String confirmPassword, String domanda, String risposta)	Metodo per registrare l'utente al sito.
login(String email, String password)	Metodo per autenticare l'utente.
getDatiUtente(String email)	Metodo per recuperare i dati del proprio profilo.
getUtentiDaApprovare()	Metodo per recuperare gli utenti da approvare.
accettaUtente(String email)	Metodo per accettare un utente.
rifiutaUtente(String email)	Metodo per rifiutare un utente.
recuperaDomanda(String email)	Metodo per ottenere la domanda di sicurezza relativa a un utente.
verificaRispostaSicurezza(String email, String risposta)	Metodo per verificare la risposta di sicurezza.
resetPassword(String email, String rispostaSicurezza, String nuovaPassword, String confermaPassword)	Metodo per cambiare la password.
isNomeValido(String testo)	Metodo per verificare che il campo nome sia valido.
isCodiceFiscaleValido(String cf)	Metodo per verificare che il campo codice fiscale sia valido.
isEmailFormatoValido(String email)	Metodo per verificare che il campo email sia valido.
isPasswordValida(String password)	Metodo per verificare che il campo password sia valido.

Gestione Libri

Operazione	Descrizione
aggiungiLibro(String titolo, String autore, String genere, String copieStr, String dataPubStr, String copertina, String descrizione)	Metodo per aggiungere un libro.
aggiornaDisponibilita(String idStr, String quantitaStr)	Metodo aggiornare il numero di copie di un libro.
rimuoviLibro(String idStr)	Metodo per rimuovere un libro dal catalogo.
getCatalogoCompleto()	Metodo per recuperare il catalogo completo.
getDettaglioLibro(int id)	Metodo per recuperare i dettagli di un libro.

Gestione Prestiti

Operazione	Descrizione
getPrenotati()	Metodo per ottenere i prestiti prenotati.
getAttivi()	Metodo per ottenere i prestiti attivi.
getRestituiti()	Metodo per ottenere i prestiti restituiti.
prenotaLibro(String emailUtente, String idLibroStr, String dataRitiroStr)	Metodo per prenotare un libro.
approvaRichiestaPrestito(String idStr)	Metodo per approvare una richiesta di prestito.
rifiutaRichiestaPrestito(String idStr, String motivazione)	Metodo per rifiutare una richiesta di prestito.
confermaRitiro(String idStr)	Metodo per confermare il ritiro di un prestito.
annullaPrestito(String idStr, String motivazione)	Metodo per annullare un prestito.
registraRestituzione(String idStr)	Metodo per registrare una restituzione.
getStoricoUtente(String email)	Metodo per ottenere lo storico di un utente.
getRichiesteInAttesa()	Metodo per ottenere le richieste in attesa.

Gestione Recensioni

Operazione	Descrizione
aggiungiRecensione(String emailUtente, String idLibroStr, String testo, String votoStr)	Metodo per aggiungere una recensione.
getRecensioniPubbliche(int idLibro)	Metodo per ottenere le recensioni pubbliche(non nascoste).
getRecensioniPerModeratore(int idLibro)	Metodo per ottenere le recensioni(moderatore).
getMappaRecensioniPerStorico(String email, List<Prestito> storico)	Metodo per ottenere le recensioni di un utente nello storico.
deleteRecensioneUtente(String emailUtente, String idLibroStr)	Metodo per rimuovere una recensione(utente).
deleteRecensioneModeratore(String idRecensioneStr)	Metodo per rimuovere una recensione(moderatore).
impostaVisibilita(String idStr, boolean visibile)	Metodo per cambiare lo stato della visibilità.

Gestione Segnalazioni

Operazione	Descrizione
segnalaRecensione(String idRecensioneStr, String emailUtente, String motivo)	Metodo per aprire una segnalazione relativa a una recensione.
getTutteLeSegnalazioni()	Metodo per ottenere tutte le segnalazioni.
chiudiSegnalazione(String idStr, String note)	Metodo per chiudere una segnalazione.